

Week 1 Report



BLUE TEAM INTERNSHIP Lab Setup

Prepared by: Hamad Ali

Prepared for: Zia Ullah

SOC Analyst Training Report

Wazuh Manager Installation and SIEM Deployment

1. Introduction

The main focus was deploying the Wazuh SIEM platform

Wazuh was selected as the primary security monitoring solution for the SOC lab because it provides capabilities for collecting logs, detecting security events, generating alerts, and monitoring endpoints.

The Wazuh architecture consists of three main components:

Wazuh Manager

Wazuh Indexer

Wazuh Dashboard

These components work together to collect endpoint data, process security events, store information, and provide an interface for analysts to investigate alerts.

2. Objectives Completed

The main objectives completed :

Installed Wazuh SIEM stack

Deployed Wazuh Manager

Installed Wazuh Indexer

Installed Wazuh Dashboard

Verified all Wazuh services

Accessed the Wazuh web interface

Explored the dashboard structure and main sections

3. Wazuh SIEM Installation

The official Wazuh installation method was used for deployment.

The quickstart installation approach was selected because it automatically installs and configures all required components for a single-node environment.

The installation process included:

Downloading required packages

Installing Wazuh Manager

Installing Wazuh Indexer

Installing Wazuh Dashboard

Applying default configurations

4. Wazuh Components Overview

4.1 Wazuh Manager

The Wazuh Manager is the core component responsible for:

Receiving logs from agents

Applying detection rules

Generating security alerts

Managing endpoint communication

4.2 Wazuh Indexer

The Indexer stores collected security information in a searchable format.

Its purpose:

Store logs

Store alert records

Allow fast searching and analysis

4.3 Wazuh Dashboard

The Dashboard provides the graphical interface used by security analysts.

It allows:

Viewing alerts

Monitoring agents

Investigating events

Reviewing security information

5. Service Verification

After installation, all Wazuh services were checked to confirm successful deployment.

Commands used:

Wazuh Manager

```
sudo systemctl status wazuh-manager
```

Result:

Service status showed:

Active: active (running)

```
■ wazuh-manager.service - Wazuh manager
   Loaded: loaded (/usr/lib/systemd/system/wazuh-manager.service; enabled; preset: disabled)
   Active: active (running) since Sat 2026-06-27 17:52:45 UTC; 12s ago
   Process: 2432 ExecStart=/usr/bin/env /var/ossec/bin/wazuh-control start (code=exited, status=0/
   Tasks: 218 (limit: 9478)
   Memory: 312.0M
   CPU: 1min 2.328s
   CGroup: /system.slice/wazuh-manager.service
           └─2508 /var/ossec/framework/python/bin/python3 /var/ossec/api/scripts/wazuh_apid.py
             └─2509 /var/ossec/framework/python/bin/python3 /var/ossec/api/scripts/wazuh_apid.py
               └─2510 /var/ossec/framework/python/bin/python3 /var/ossec/api/scripts/wazuh_apid.py
                 └─2513 /var/ossec/framework/python/bin/python3 /var/ossec/api/scripts/wazuh_apid.py
                   └─2516 /var/ossec/framework/python/bin/python3 /var/ossec/api/scripts/wazuh_apid.py
                     └─2564 /var/ossec/bin/wazuh-authd
                       └─2577 /var/ossec/bin/wazuh-db
                         └─2598 /var/ossec/bin/wazuh-execd
                           └─2635 /var/ossec/bin/wazuh-analysd
                             └─2648 /var/ossec/bin/wazuh-syscheckd
                               └─2671 /var/ossec/bin/wazuh-remoted
                                 └─2713 /var/ossec/bin/wazuh-logcollector
                                   └─2727 /var/ossec/bin/wazuh-monitord
                                     └─2742 /var/ossec/bin/wazuh-modulesd
                                       └─3194 sh -c "rpm -qa | grep -E 'python.*' | xargs -I {} rpm -ql {} | grep -E '\\.egg-
                                         └─3197 xargs -I {} rpm -ql {}
                                           └─3198 grep -E "\\.(egg-info|\\.dist-info)$"
                                             └─3630 rpm -ql python3-systemd-235-51.amzn2023.0.2.x86_64
                                               └─3631 sshd -T

Jun 27 17:52:38 wazuh-server env[2432]: wazuh-logcollector: Process 2715 not used by Wazuh, removin
Jun 27 17:52:38 wazuh-server env[2432]: Started wazuh-logcollector...
Jun 27 17:52:39 wazuh-server env[2432]: wazuh-monitord: Process 2727 not used by Wazuh, removing...
Jun 27 17:52:39 wazuh-server env[2432]: Started wazuh-monitord...
Jun 27 17:52:42 wazuh-server env[2432]: wazuh-modulesd: Process 2739 not used by Wazuh, removing...
Jun 27 17:52:43 wazuh-server env[2739]: 2026/06/27 17:52:42 wazuh-modulesd:router: INFO: Loaded rou
Jun 27 17:52:43 wazuh-server env[2739]: 2026/06/27 17:52:42 wazuh-modulesd:content_manager: INFO: L
Jun 27 17:52:43 wazuh-server env[2739]: 2026/06/27 17:52:42 wazuh-modulesd:inventory-harvester: INF
```

lines 1-36

Wazuh Indexer

```
sudo systemctl status wazuh-indexer
```

Result:

Active: active (running)

```
[root@wazuh-server ~]# systemctl status wazuh-indexer
■ wazuh-indexer.service - wazuh-indexer
   Loaded: loaded (/usr/lib/systemd/system/wazuh-indexer.service; enabled; preset: disabled)
   Active: active (running) since Sat 2026-06-27 17:53:56 UTC; 1min 8s ago
     Docs: https://documentation.wazuh.com
    Main PID: 2092 (java)
      Tasks: 95 (limit: 9478)
     Memory: 4.6G
        CPU: 4min 55.742s
    CGroup: /system.slice/wazuh-indexer.service
            └─2092 /usr/share/wazuh-indexer/jdk/bin/java -Xshare:auto -Dopensearch.networkaddress.>

Jun 27 17:51:54 wazuh-server systemd-entrypoint[2092]: WARNING: A terminally deprecated method in j>
Jun 27 17:51:54 wazuh-server systemd-entrypoint[2092]: WARNING: System::setSecurityManager has been>
Jun 27 17:51:54 wazuh-server systemd-entrypoint[2092]: WARNING: Please consider reporting this to t>
Jun 27 17:51:54 wazuh-server systemd-entrypoint[2092]: WARNING: System::setSecurityManager will be >
Jun 27 17:51:57 wazuh-server systemd-entrypoint[2092]: Jun 27, 2026 5:51:57 PM sun.util.locale.prov>
Jun 27 17:51:57 wazuh-server systemd-entrypoint[2092]: WARNING: COMPAT locale provider will be remo>
Jun 27 17:52:00 wazuh-server systemd-entrypoint[2092]: WARNING: A terminally deprecated method in j>
Jun 27 17:52:00 wazuh-server systemd-entrypoint[2092]: WARNING: System::setSecurityManager has been>
Jun 27 17:52:00 wazuh-server systemd-entrypoint[2092]: WARNING: Please consider reporting this to t>
Jun 27 17:52:00 wazuh-server systemd-entrypoint[2092]: WARNING: System::setSecurityManager will be >
lines 1-21/21 (END)
```

Wazuh Dashboard

```
sudo systemctl status wazuh-dashboard
```

Result:

Active: active (running)

```
[root@wazuh-server ~]# systemctl status wazuh-dashboard
■ wazuh-dashboard.service - wazuh-dashboard
   Loaded: loaded (/etc/systemd/system/wazuh-dashboard.service; enabled; preset: disabled)
   Active: active (running) since Sat 2026-06-27 17:50:15 UTC; 3min 47s ago
    Main PID: 1763 (node)
      Tasks: 11 (limit: 9478)
     Memory: 264.8M
        CPU: 55.948s
    CGroup: /system.slice/wazuh-dashboard.service
            └─1763 /usr/share/wazuh-dashboard/node/bin/node --no-warnings --max-http-header-size=6>

Jun 27 17:53:40 wazuh-server opensearch-dashboards[1763]: {"type":"log","@timestamp":"2026-06-27T17>
Jun 27 17:53:42 wazuh-server opensearch-dashboards[1763]: {"type":"log","@timestamp":"2026-06-27T17>
Jun 27 17:53:45 wazuh-server opensearch-dashboards[1763]: {"type":"log","@timestamp":"2026-06-27T17>
Jun 27 17:53:47 wazuh-server opensearch-dashboards[1763]: {"type":"log","@timestamp":"2026-06-27T17>
Jun 27 17:53:50 wazuh-server opensearch-dashboards[1763]: {"type":"log","@timestamp":"2026-06-27T17>
Jun 27 17:53:52 wazuh-server opensearch-dashboards[1763]: {"type":"log","@timestamp":"2026-06-27T17>
Jun 27 17:53:55 wazuh-server opensearch-dashboards[1763]: {"type":"log","@timestamp":"2026-06-27T17>
Jun 27 17:54:00 wazuh-server opensearch-dashboards[1763]: {"type":"log","@timestamp":"2026-06-27T17>
Jun 27 17:54:00 wazuh-server opensearch-dashboards[1763]: {"type":"log","@timestamp":"2026-06-27T17>
Jun 27 17:54:03 wazuh-server opensearch-dashboards[1763]: {"type":"log","@timestamp":"2026-06-27T17>
lines 1-20/20 (END)
```

6. Accessing Wazuh Dashboard

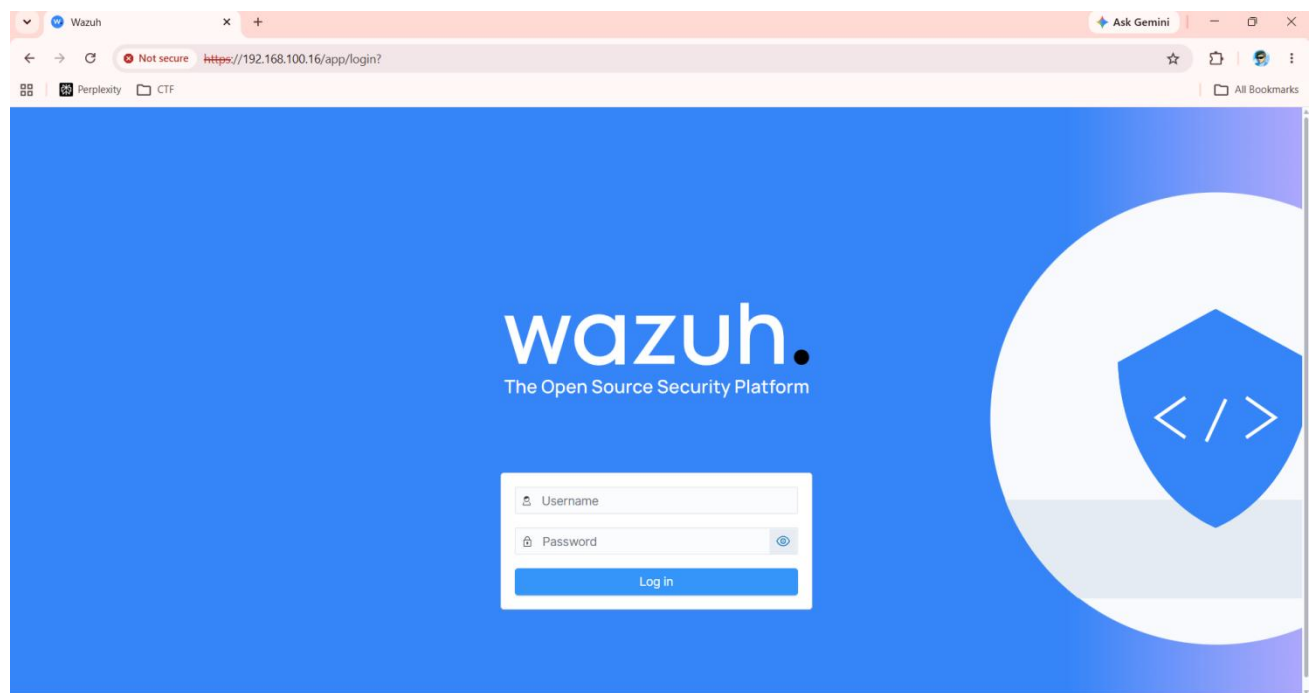
After successful installation, the Wazuh Dashboard was accessed through a web browser.

URL format:

`https://192.168.100.16`

The login credentials generated during installation were used to access the dashboard.

After login, the default password was changed for security purposes.



7. Dashboard Exploration

After accessing the dashboard, different sections were reviewed:

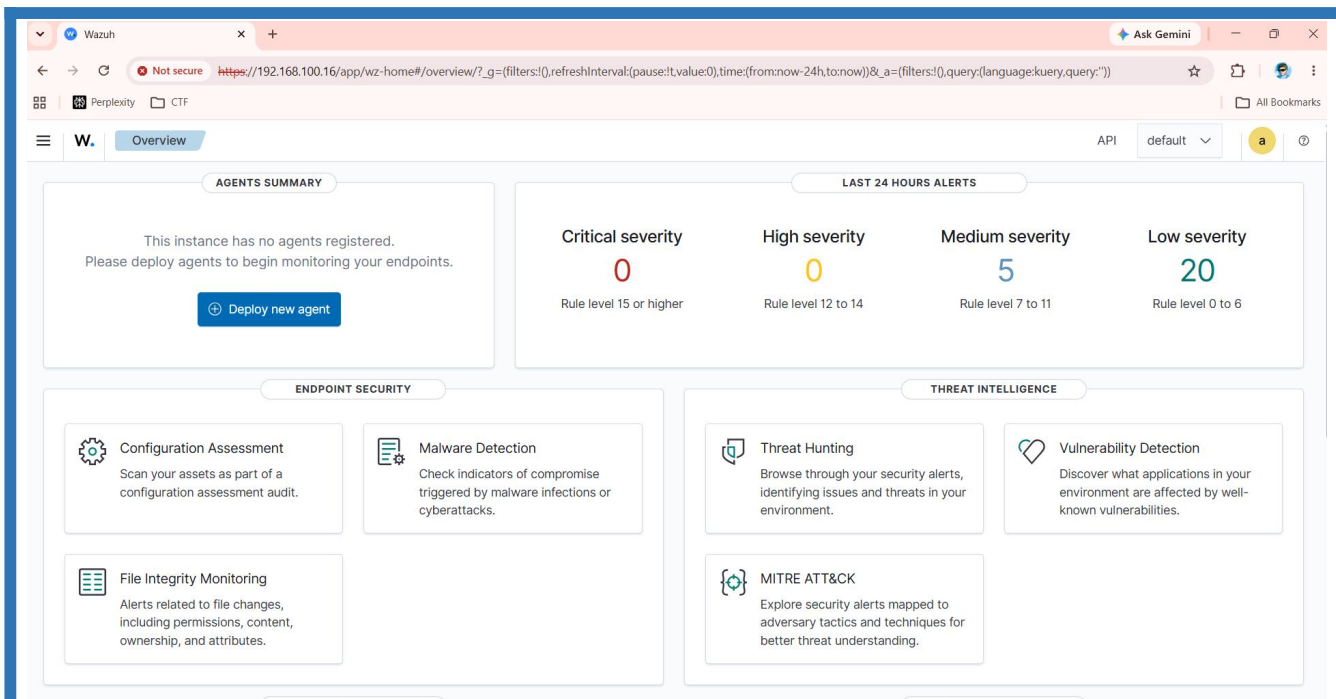
Overview

Used for:

General security information

System status

Monitoring overview



Agents Section

Used for:

Viewing connected endpoints

Checking agent status

Managing monitored systems

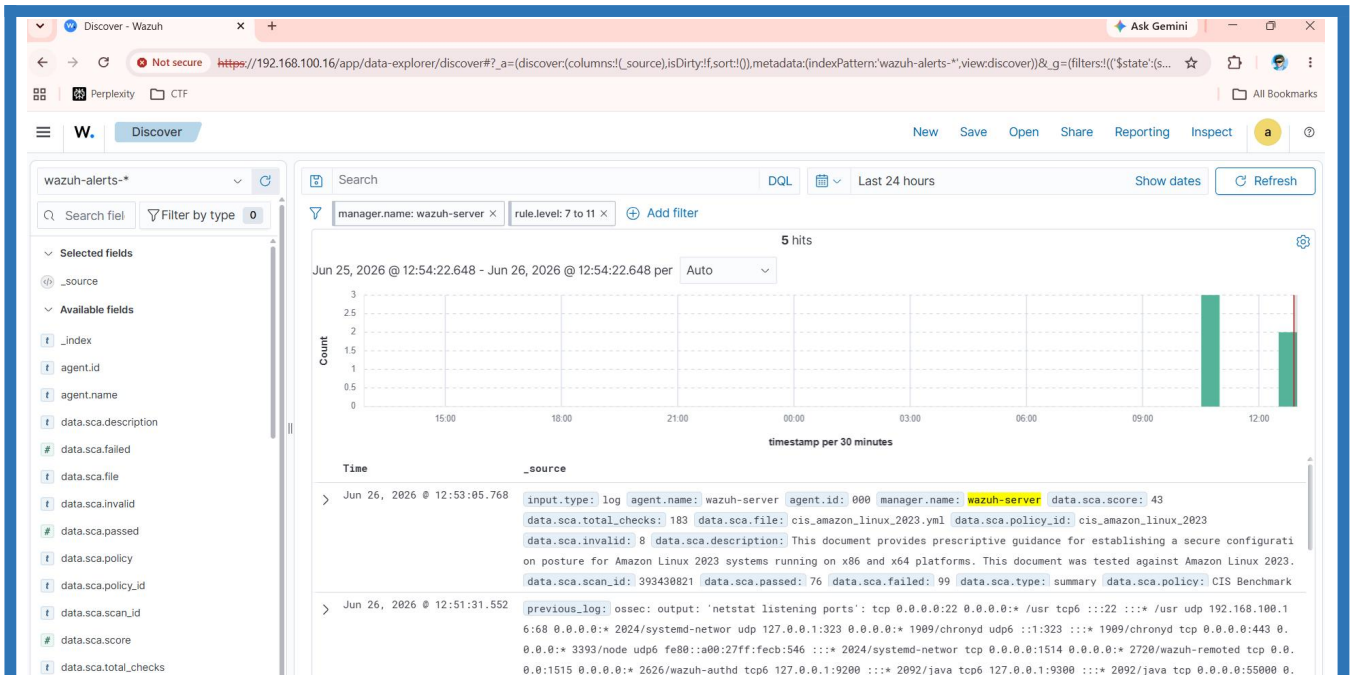
Security Events

Used for:

Reviewing alerts

Investigating detected activities

Understanding event details



8. Learning Outcome

After completing , I gained practical understanding of SIEM architecture and Wazuh deployment.

I learned:

How a SIEM collects and processes logs

The role of Manager, Indexer, and Dashboard

How analysts access and investigate security events

How to verify SIEM services

The Wazuh environment was successfully prepared and is ready for endpoint agent integration in the next phase.

1. Introduction

The main objective was connecting endpoint machines with the Wazuh SIEM server.

After successfully deploying the Wazuh Manager, Indexer, and Dashboard, the next step was to install Wazuh agents on monitored systems.

A Wazuh agent is a lightweight service installed on endpoints that collects system logs, security events, and activity information, then sends this data to the Wazuh Manager for analysis.

The agents were deployed on:

Kali Linux

Windows 10

This created a complete monitoring flow:

Endpoint



Wazuh Agent



Wazuh Manager



Wazuh Dashboard



Security Analyst Investigation

2. Objectives Completed

The main objectives completed :

Generated agent installation commands from Wazuh Dashboard

Installed Wazuh agent on Kali Linux

Installed Wazuh agent on Windows 10

Connected both endpoints with Wazuh Manager

Verified agent status in dashboard

Reviewed agent configuration file

3. Wazuh Agent Deployment Process

The Wazuh Dashboard agent deployment wizard was used to generate installation commands.

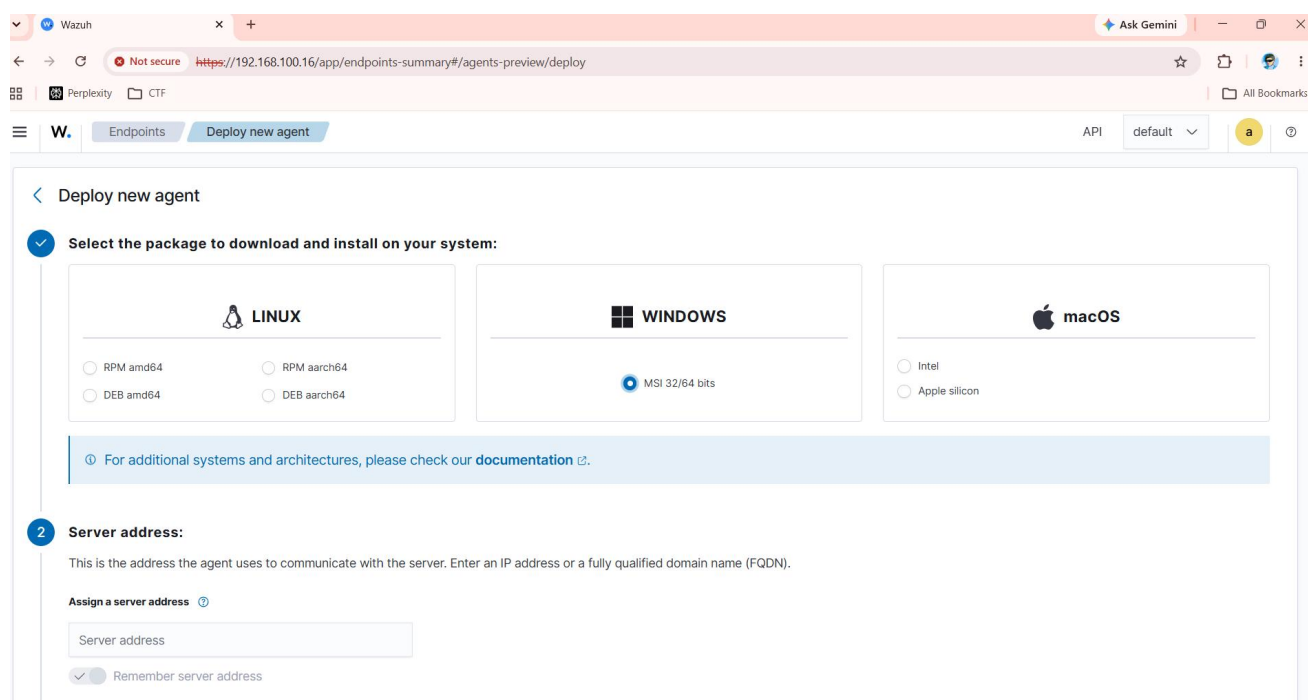
The wizard automatically provided:

Correct operating system package

Manager IP address

Agent registration details

Using the built-in wizard reduced configuration mistakes and ensured that each agent was correctly connected to the Wazuh Manager.



4. Kali Linux Agent Installation

The Wazuh agent was installed on the Kali Linux virtual machine.

The generated installation command from the dashboard was executed.

After installation, the agent service was started.

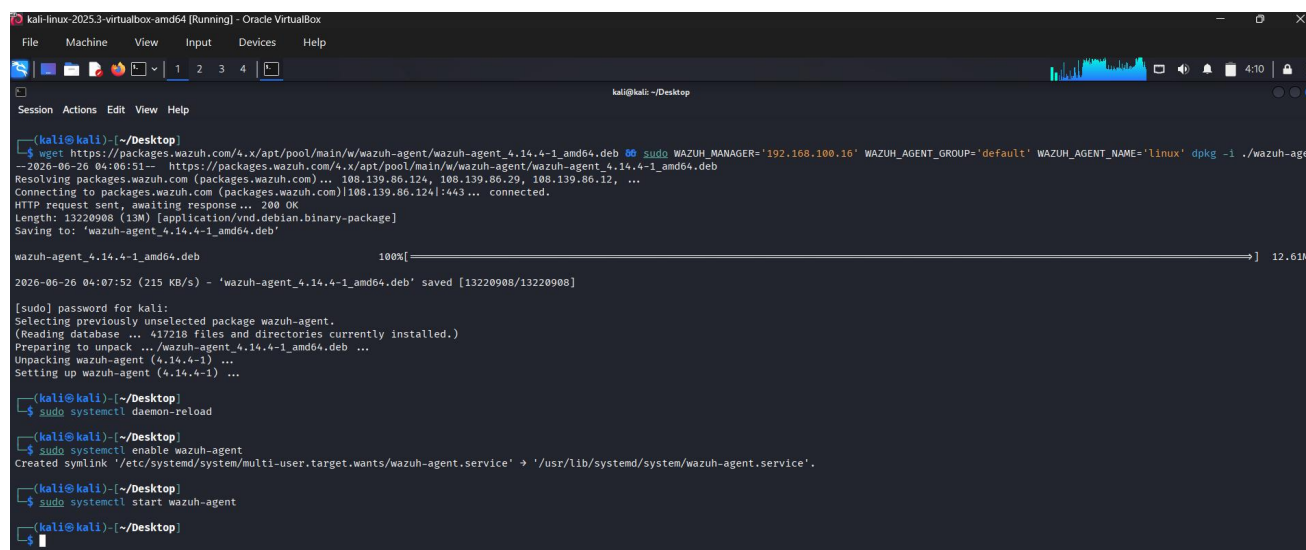
Command used:

```
sudo systemctl start wazuh-agent
```

Service status was checked:

```
sudo systemctl status wazuh-agent
```

The agent successfully connected to the Wazuh Manager.



```
kali@kali: ~/Desktop
$ wget https://packages.wazuh.com/4.x/apt/pool/main/w/wazuh-agent/wazuh-agent_4.14.4-1_amd64.deb
--2026-06-26 04:06:51-- https://packages.wazuh.com/4.x/apt/pool/main/w/wazuh-agent/wazuh-agent_4.14.4-1_amd64.deb
Resolving packages.wazuh.com (packages.wazuh.com)... 108.139.86.124, 108.139.86.29, 108.139.86.12, ...
Connecting to packages.wazuh.com (packages.wazuh.com)[108.139.86.124]:443 ... connected.
HTTP request sent, awaiting response... 200 OK
Length: 13220908 (13M) [application/vnd.debian.binary-package]
Saving to: 'wazuh-agent_4.14.4-1_amd64.deb'

wazuh-agent_4.14.4-1_amd64.deb      100%[=====] 12.61M

2026-06-26 04:07:52 (215 KB/s) - 'wazuh-agent_4.14.4-1_amd64.deb' saved [13220908/13220908]

[sudo] password for kali:
Selecting previously unselected package wazuh-agent.
(Reading database ... 417218 files and directories currently installed.)
Preparing to unpack .../wazuh-agent_4.14.4-1_amd64.deb ...
Unpacking wazuh-agent (4.14.4-1) ...
Setting up wazuh-agent (4.14.4-1) ...

(kali@kali)~$ sudo systemctl daemon-reload
(kali@kali)~$ sudo systemctl enable wazuh-agent
Created symlink '/etc/systemd/system/multi-user.target.wants/wazuh-agent.service' → '/usr/lib/systemd/system/wazuh-agent.service'.
(kali@kali)~$ sudo systemctl start wazuh-agent
(kali@kali)~$
```

5. Windows 10 Agent Installation

The Wazuh agent was installed on the Windows 10 endpoint.

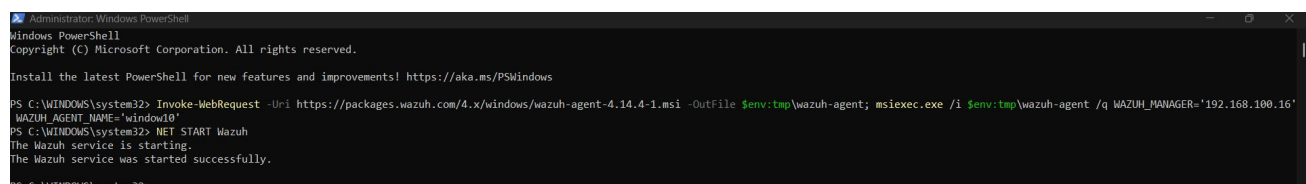
The deployment command generated from the Wazuh Dashboard was executed using administrator PowerShell.

After installation, the Wazuh service was started.

The service was verified from Windows Services.

Result:

The Windows endpoint successfully registered with the Wazuh Manager.



```
Administrator: Windows PowerShell
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

Install the latest PowerShell for new features and improvements! https://aka.ms/PSWindows

PS C:\WINDOWS\system32> Invoke-WebRequest -Uri https://packages.wazuh.com/4.x/windows/wazuh-agent-4.14.4-1.msi -OutFile $env:tmp\wazuh-agent; msixexec.exe /i $env:tmp\wazuh-agent /q WAZUH_MANAGER="192.168.100.16" WAZUH_AGENT_NAME="window10"
PS C:\WINDOWS\system32> NET START Wazuh
The Wazuh service is starting.
The Wazuh service was started successfully.
PS C:\WINDOWS\system32>
```

6. Agent Status Verification

After both installations, the Wazuh Dashboard was opened and the Agents section was checked.

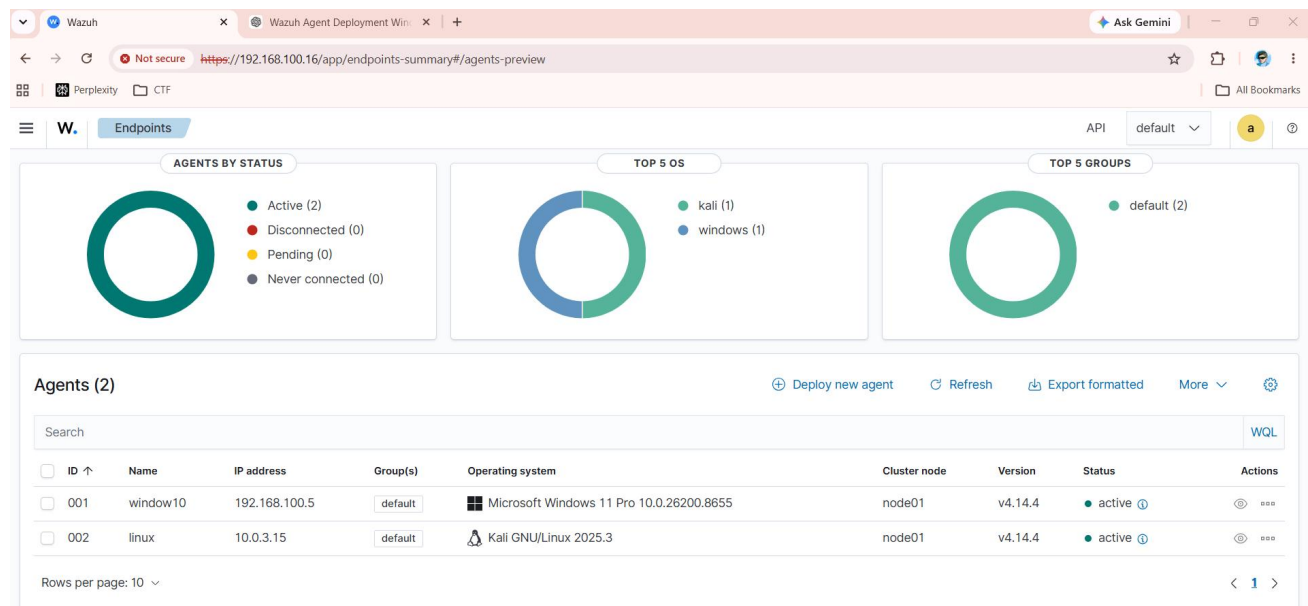
Both endpoints appeared successfully:

Kali Linux:

Status: **Active**

Windows 10:

Status: **Active**



7. Learning Outcome

After completing , I understood how endpoint monitoring works in a SOC environment.

I learned:

How agents communicate with a SIEM server

How endpoint logs are collected

Basic troubleshooting of agent connectivity

The Wazuh environment was successfully expanded from a single server into a complete monitored SOC lab with multiple endpoints.

Wazuh Dashboard Navigation and Alert Triage

1. Introduction

Now focused on moving from lab setup and deployment into practical SOC analysis.

After completing the Wazuh installation and connecting endpoint agents, the next step was learning how to use the dashboard for monitoring, investigation, and alert triage.

Alert triage is an important SOC analyst skill where security events are reviewed, prioritized, and analyzed to determine whether they represent real threats, false positives, or normal system activity.

2. Objectives Completed

The main objectives completed :

Explored Wazuh Dashboard interface

Reviewed Security Alerts section

Analyzed individual alerts

Understood Wazuh rule severity levels

Used Threat Hunting features

Reviewed MITRE ATT&CK mappings

Performed basic alert investigation

Created an understanding of normal environment behavior

3. Wazuh Dashboard Exploration

The Wazuh Dashboard was opened and different sections were reviewed.

The main focus areas were:

Security Alerts

The Security Alerts section was analyzed to understand:

Alert description

Generated time

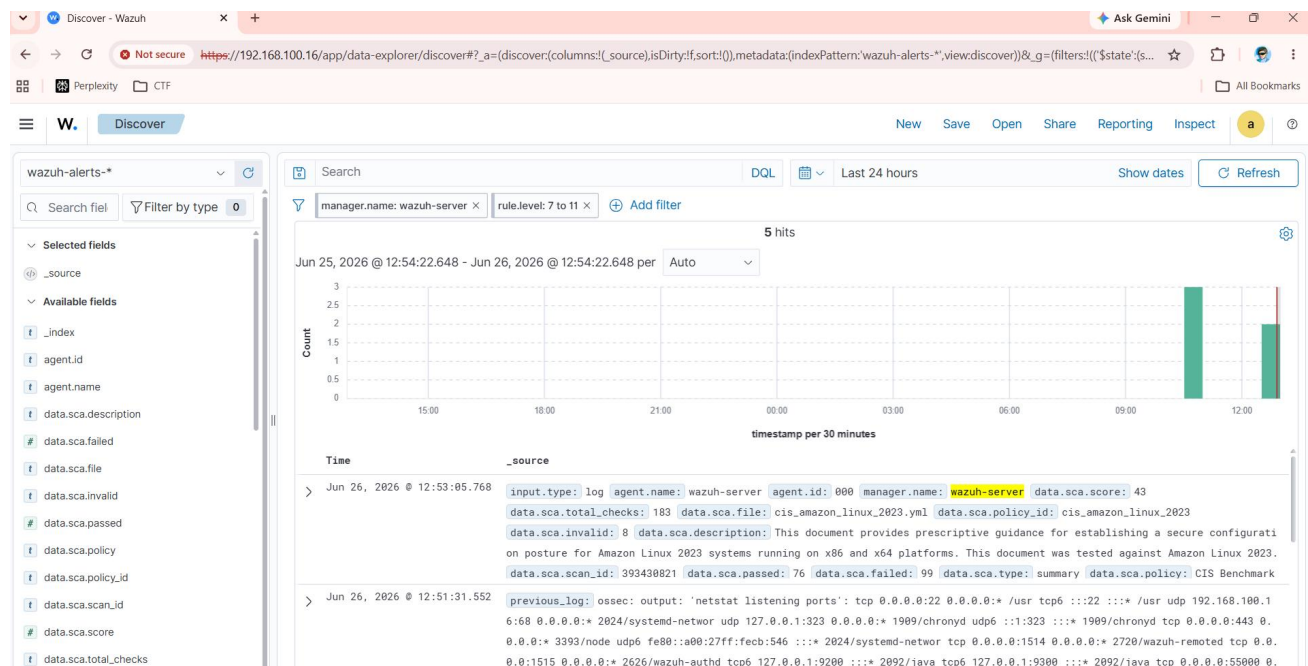
Source agent

Rule ID

Severity level

Event details

Each alert was opened individually to understand what activity triggered the detection.



4. Alert Severity Understanding

Wazuh uses rule levels from 0 to 15 to indicate event importance.

Severity classification:

Rule Level	Category
0 - 3	Informational
4 - 7	Low Severity
8 - 11	Medium Severity
12 - 15	High / Critical

During analysis, I learned that severity alone does not determine whether an event is dangerous.

The context, frequency, source, and behavior of the event must also be considered.

5. Threat Hunting Practice

The Threat Hunting module was explored to practice searching and filtering security events.

Filters tested:

Agent name

Time range

Rule level

Specific event fields

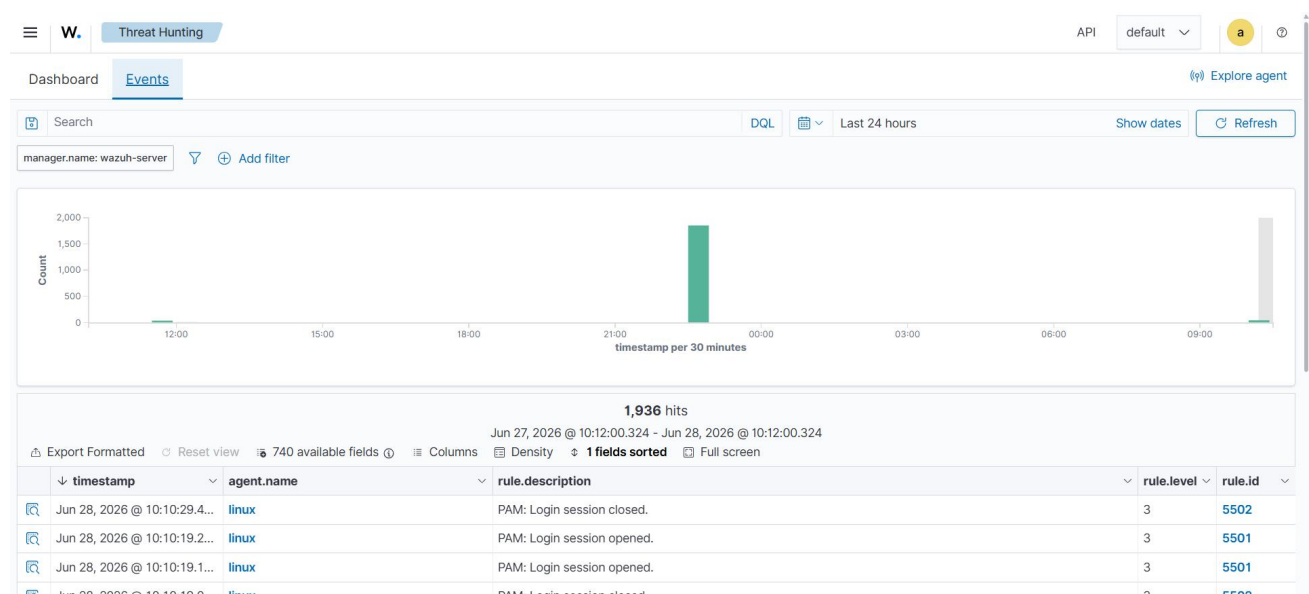
These filters helped in quickly locating relevant events instead of manually reviewing every alert.

Example searches included:

Viewing alerts from a specific endpoint

Finding high severity events

Checking repeated authentication failures



6. MITRE ATT&CK Mapping

The MITRE ATT&CK module was reviewed to understand how security events relate to attacker techniques.

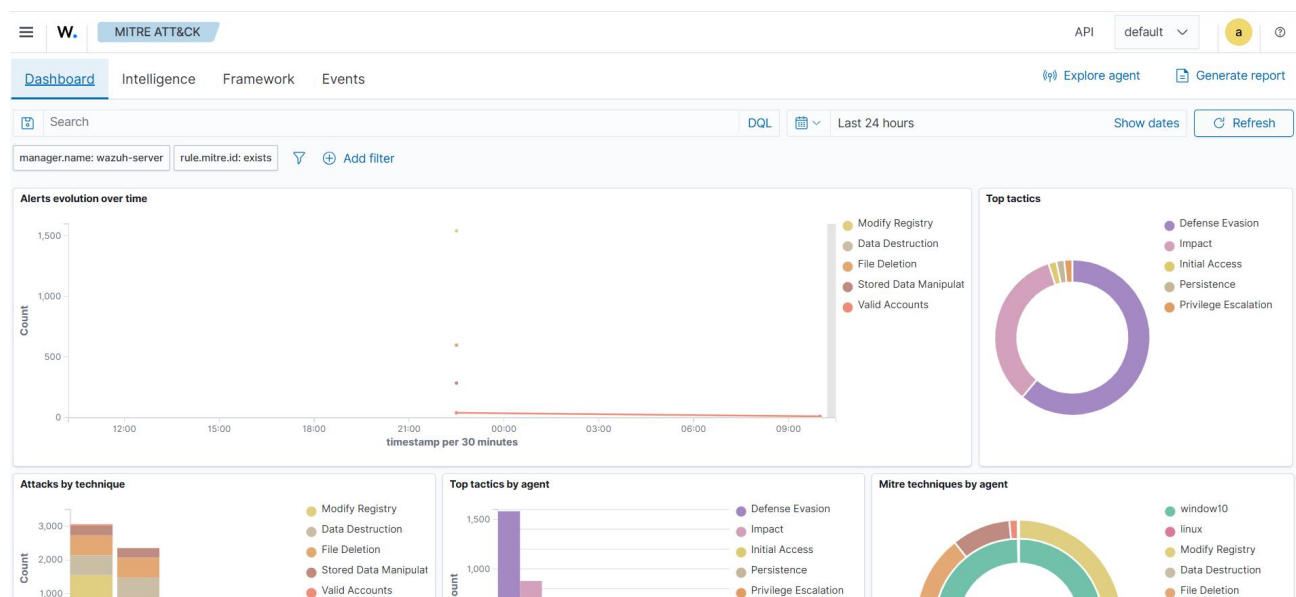
MITRE ATT&CK provides a structured framework for categorizing adversary behavior.

It helps analysts understand:

How an attacker may operate

Which technique is being detected

What defensive actions can be applied



7. Alert Triage Process

A basic alert triage process was followed:

Step 1: Alert Identification

Reviewed:

Rule ID

Severity

Source agent

Timestamp

Step 2: Event Analysis

Checked:

What activity occurred

Which system generated the event

Whether the behavior was expected

Step 3: Decision Making

The alert was classified as:

Normal activity

Requires monitoring

Potential security concern

8. Test Alert Investigation

Previously generated failed RDP Login attempts were reviewed inside the dashboard.

The investigation included:

Checking source machine

Reviewing authentication event details

Confirming the time of activity

Understanding the triggered rule

The alert confirmed that the complete detection pipeline was working:

Endpoint Activity



Agent Collection



Wazuh Manager



Detection Rule



Dashboard Alert

Discover wazuh-alerts-4.x-2026.06.27#3cwsB58BuXXthWzWhjnm	
Table	JSON
@timestamp	Jun 27, 2026 @ 08:48:48.488
_index	wazuh-alerts-4.x-2026.06.27
agent.id	001
agent.ip	192.168.100.5
agent.name	window10
data.win.eventdata.authenticationPackageName	NTLM
data.win.eventdata.elevatedToken	%%1843
data.win.eventdata.imPERSONationLevel	%%1833
data.win.eventdata.ipAddress	192.168.15.4
data.win.eventdata.ipPort	0
data.win.eventdata.keyLength	128
data.win.eventdata.lmPackageName	NTLM V2
data.win.eventdata.logonGuid	{00000000-0000-0000-0000-000000000000}
data.win.eventdata.logonProcessName	NtLmSsp
data.win.eventdata.logonType	3
data.win.eventdata.processId	0x0
data.win.eventdata.subjectLogonId	0x0
data.win.eventdata.subjectUserSid	S-1-0-0
data.win.eventdata.subjectUserName	REFERENCE

9. Learning Outcome

After completing , I developed basic SOC analyst skills including:

Navigating SIEM dashboards

Reviewing security alerts

Understanding severity levels

Filtering events efficiently

Performing basic alert triage

Connecting alerts with MITRE ATT&CK techniques

I learned that effective monitoring depends on understanding normal system behavior first. Once a baseline is created, unusual activity becomes easier to identify.

File Integrity Monitoring (FIM) Configuration in Wazuh

Objective

The objective was to learn and implement **File Integrity Monitoring (FIM)** using Wazuh. The purpose of File Integrity Monitoring is to continuously monitor important files and directories for unauthorized modifications, creations, or deletions. This helps security analysts detect suspicious activities such as malware infections, unauthorized configuration changes, privilege escalation, and persistence mechanisms used by attackers.

Introduction to File Integrity Monitoring (FIM)

File Integrity Monitoring (FIM) is one of the most important security monitoring features provided by Wazuh. It continuously observes selected **files and directories** and detects any change made to them.

Whenever a monitored file is:

- Created
- Modified
- Deleted
- Renamed
- Had its permissions changed

the Wazuh agent immediately records the event and sends it to the Wazuh Manager. The manager processes the event and displays an alert on the Wazuh Dashboard.

This enables SOC analysts to quickly detect unauthorized modifications that could indicate malicious activity.

Step 1: Opening the Wazuh Configuration File

The first step was to open the Wazuh agent configuration file (**ossec.conf**) on both Linux and Windows agents.

Linux Location:

`/var/ossec/etc/ossec.conf`

```
root@kali: /var/ossec/etc
```

Windows Location:

`C:\Program Files (x86)\ossec-agent\ossec.conf`

Inside this configuration file, the **syscheck** section controls all File Integrity Monitoring settings.

Name	Date modified	Type	Size
libstdc++-6.dll	3/13/2026 10:35 AM	Application extens...	2,303 K
libwazuext.dll	3/13/2026 10:35 AM	Application extens...	8,530 K
libwazushared.dll	3/13/2026 10:35 AM	Application extens...	274 K
libwinpthread-1.dll	3/13/2026 10:35 AM	Application extens...	59 K
LICENSE	3/13/2026 10:30 AM	Text Document	25 K
local_internal_options.conf	3/13/2026 10:30 AM	CONF File	1 K
manage_agents	3/13/2026 10:35 AM	Application	354 K
ossec.conf	7/1/2026 10:55 AM	CONF File	5 K

Step 2: Configuring File Integrity Monitoring

The default syscheck configuration was modified to monitor additional directories.

Linux Directories

The following directories were added:

/etc

Purpose:

- Stores Linux system configuration files.
- Critical files like passwd, shadow, ssh configuration, sudoers etc. exist here.
- Attackers often modify these files after gaining access.

Configuration:

```
<directories realtime="yes" report_changes="yes">/etc</directories>
```

The second monitored directory was:

/var/log

Purpose:

- Stores operating system logs.
- Application logs
- Authentication logs
- Security logs

Configuration:

```
<directories realtime="yes">/var/log</directories>
```

```
Session Actions Edit View Help
GNU nano 8.6
<!-- Logging -->
<logging>
  <log_format>plain</log_format>
</logging>

<!-- File Integrity Monitoring -->
<syscheck>

  <disabled>no</disabled>

  <frequency>43200</frequency>

  <directories realtime="yes" report_changes="yes">/etc</directories>
  <directories realtime="yes">/var/log</directories>

  <ignore>/etc/mtab</ignore>
  <ignore>/etc/hosts.deny</ignore>

</syscheck>
```

Windows Directory

The following directory was monitored:

C:\Windows\System32

Purpose:

This folder contains:

- Windows DLL files
- Executable files
- Drivers
- Critical operating system components

Configuration:

```
<directories realtime="yes" report_changes="yes">C:\Windows\System32</directories>
```

```

ossec.conf
Edit View
<!-- Security Configuration Assessment -->
<sca>
  <enabled>yes</enabled>
  <scan_on_start>yes</scan_on_start>
  <interval>12h</interval>
  <skip_nfs>yes</skip_nfs>
</sca>

<!-- File integrity monitoring -->
<syscheck>

  <disabled>no</disabled>

  <!-- Scan every 60 seconds -->
  <frequency>60</frequency>

  <!-- Monitor only the Downloads folder -->
  <directories realtime="yes" report_changes="yes">C:\Users\Public</directories>
  <directories realtime="yes" report_changes="yes">C:\Windows\System32</directories>

```

Step 3: Configuring Scan Frequency

The scheduled scan frequency was changed.

```
<frequency>600</frequency>
```

Explanation:

600 seconds means Wazuh performs a scheduled integrity scan every 10 minutes for directories that are not using realtime monitoring.

Realtime directories are monitored immediately using operating system filesystem notifications.

```

<frequency>600</frequency>

<directories realtime="yes" report_changes="yes">/etc</directories>
<directories realtime="yes">/var/log</directories>

<!-- File integrity monitoring -->
<syscheck>

  <disabled>no</disabled>

  <!-- Scan every 60 seconds -->
  <frequency>600</frequency>

  <!-- Monitor only the Downloads folder -->
  <directories realtime="yes" report_changes="yes">C:\Users\Public</directories>
  <directories realtime="yes" report_changes="yes">C:\Windows\System32</directories>

```

Step 4: Restarting the Wazuh Agent

After saving the configuration, the Wazuh Agent service was restarted.

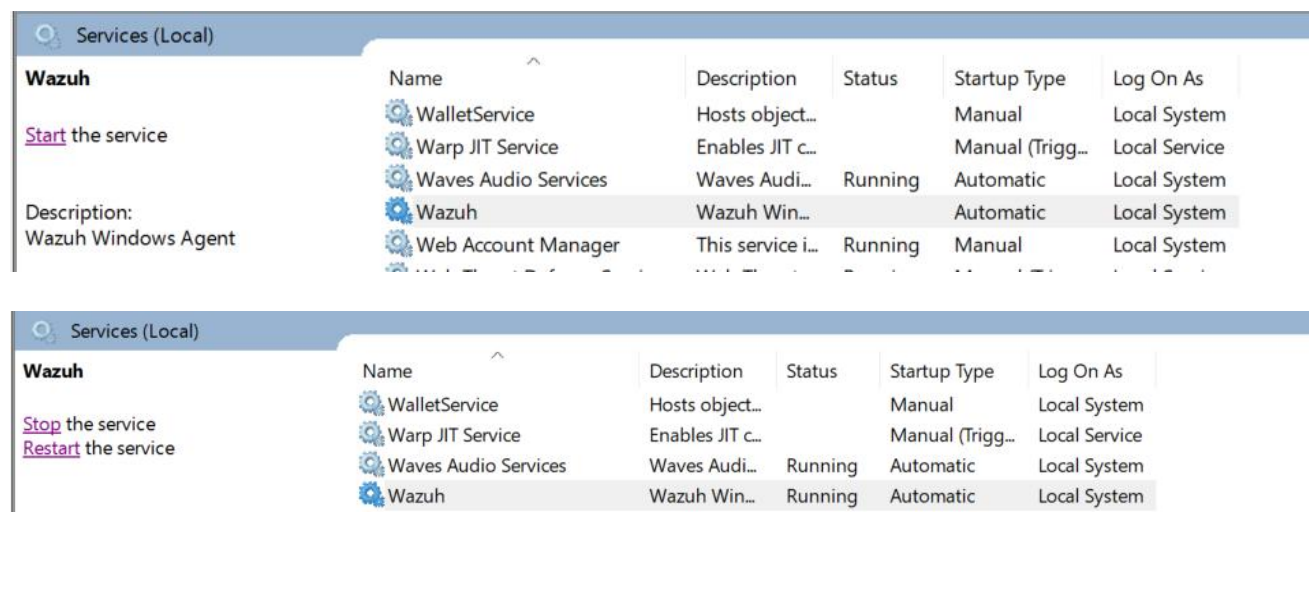
Linux Command

sudo systemctl restart wazuh-agent

```
(kali㉿kali)-[/]  
$ sudo systemctl restart wazuh-agent  
[sudo] password for kali:
```

Windows

Restarted the **Wazuh Agent** service from Windows Services.



Step 5: Verifying Agent Status

After restarting, the Wazuh Dashboard was opened.

The status of both agents was verified.

Results:

- Linux Agent → **Active**
- Windows Agent → **Active**

This confirmed that the new File Integrity Monitoring configuration had been successfully loaded.

Agents (2)

Deploy new agent

Refresh

Export formatted

More

status=active

WQL

☐	ID ↑	Name	IP address	Group(s)	Operating system	Cluster node	Version	Status	Actions
☐	003	ubuntu-agent	10.0.3.15	default	 Kali GNU/Linux 2025.3	node01	v4.14.4	● active ⓘ	 ...

Agents (1)

Deploy new agent

Refresh

Export formatted

More

status=active

WQL

☐	ID ↑	Name	IP address	Group(s)	Operating system	Cluster node	Version	Status	Actions
☐	004	windows	192.168.100.5	default	 Microsoft Windows 11 Pro 10.0.26200.8655	node01	v4.14.4	● active ⓘ	 ...

Rows per page: 10

< 1 >

Conclusion

focused on configuring File Integrity Monitoring on both Linux and Windows systems. Important directories were selected based on their security value, realtime monitoring was enabled, scan frequency was configured, and both agents were successfully restarted. Finally, both agents were verified as active in the Wazuh Dashboard, confirming that the configuration was correctly applied.

Testing and Analysis of File Integrity Monitoring Alerts

Objective

The objective was to generate File Integrity Monitoring events and analyze the alerts generated by Wazuh.

Step 1: Generating Test Events on Linux

Several file operations were performed inside the monitored directory.

A new file was created.

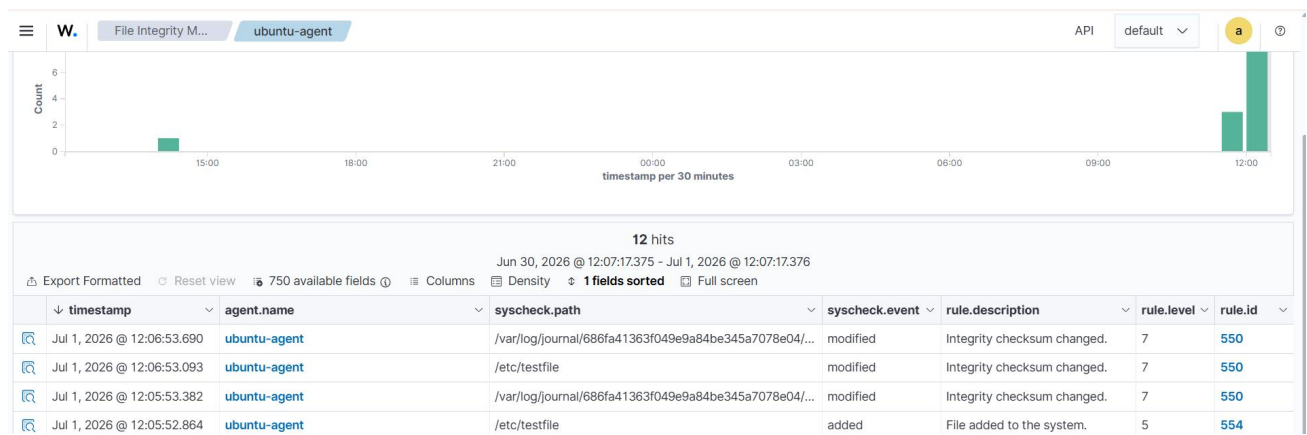
Example:

```
sudo touch /etc/testfile
```

The file was modified.

```
echo "Testing Wazuh FIM" | sudo tee -a /etc/testfile
```

Since realtime monitoring was enabled, alerts were generated immediately.



Step 2: Generating Test Events on Windows

A file was created inside:

C:\Users\Public

The file contents were modified.

Finally, the file was deleted.

Because realtime monitoring was enabled, Wazuh immediately generated alerts.

1,666 hits						
Jun 30, 2026 @ 10:55:51.589 - Jul 1, 2026 @ 10:55:51.590						
Export Formatted Reset view 750 available fields Columns Density 1 fields sorted Full screen						
timestamp	agent.name	syscheck.path	syscheck.event	rule.description	rule.level	rule.id
Jul 1, 2026 @ 10:55:41.010	windows	c:\users\public\documents\testfile.txt	added	File added to the system.	5	554
Jul 1, 2026 @ 10:55:41.001	windows	c:\users\public\documents\new text document.txt	deleted	File deleted.	7	553
Jul 1, 2026 @ 10:55:36.439	windows	c:\users\public\documents\new text document.txt	added	File added to the system.	5	554

Step 3: Viewing Alerts in Wazuh Dashboard

The Wazuh Dashboard was opened.

The Security Events page displayed multiple File Integrity Monitoring alerts.

Each alert contained valuable forensic information including:

- File path
- Event type
- Agent name
- Timestamp
- User
- Rule description

Screenshot Placeholder

[Insert Screenshot: FIM Alerts Dashboard]

Step 4: Event Analysis

The generated alerts were carefully analyzed.

The following information was extracted from each alert.

Document Details

[View surrounding documents](#)[View single document](#)

Table JSON

t _index	wazuh-alerts-4.x-2026.07.01
t agent.id	003
t agent.ip	10.0.3.15
t agent.name	ubuntu-agent
t decoder.name	syscheck_integrity_changed
t full_log	File '/etc/testfile' modified Mode: realtime Changed attributes: size,mtime,md5,sha1,sha256 Size changed from '0' to '18' Old modification time was: '1782889550', now it is '1782889610' Old md5sum was: 'd41d8cd98f00b204e9800998ecf8427e' New md5sum is: 'dd93cae1023b60090d28861ec95b5c82'
t id	1782889613.817251
t input.type	log
t location	syscheck
t manager.name	wazuh-server
t rule.description	Integrity checksum changed.
# rule.firedtimes	6
t rule.gdpr	II_5.1.f

Document Details

[View surrounding documents](#)[View single document](#)

t rule.groups	ossec, syscheck, syscheck_entry_modified, syscheck_file
t rule.hipaa	164.312.c.1, 164.312.c.2
t rule.id	550
# rule.level	7
rule.mail	false
t rule.mitre.id	T1565.001
t rule.mitre.tactic	Impact
t rule.mitre.technique	Stored Data Manipulation
t rule.nist_800_53	SI.7
t rule.pci_dss	11.5
t rule.tsc	PI1.4, PI1.5, CC6.1, CC6.8, CC7.2, CC7.3
t syscheck.changed_attributes	size, mtime, md5, sha1, sha256
t syscheck.diff	0a1 > Testing Wazuh FIM
t syscheck.event	modified
t syscheck.gid_after	0
t syscheck.gname_after	root
t syscheck.inode_after	4982379
t syscheck.md5_after	dd93cae1023b60090d28861ec95b5c82

Investigation Analysis

If these alerts had appeared in a real production environment without prior knowledge, the first step would be to determine whether the changes were authorized. The user account responsible

for the activity would be verified, followed by checking system logs, authentication records, and recent administrative actions. If the modifications were not approved, the affected system would be isolated from the network to prevent further compromise. Additional forensic analysis would then be performed to identify the source of the activity, determine whether malware or unauthorized access was involved, and assess the overall impact. Finally, the incident would be documented and escalated according to the organization's incident response procedures.